below for their actual number due to inevitable variability LLMs. That's fine for our goals since we do not want to enumerate all possible $\mathcal{C}$, but rather want to demonstrate the phenomenon.

We use the default temperature (and other hyperparameters) by providers and try to ensure reproducibility by setting seed for generation where applicable.

As our understanding judge U we employ a rule-based judge $U(W) = \min_{w,a}(D_L(w, a)) < O$. That is, we clean up $\mathcal{M}(I_L)$ and split it into words $\{w\}$ and then find the minimum Levenstein distance D_L[2] between $\{w\}$ and a set of predefined anchors $\{a\}$. The phrase is considered to be understood if this minimum distance is less than the predefined offset $O = 4$.

We use such metric because it turned out to be a useful proxy for success in our initial experiments: we saw that models sometimes produce texts like `Abra cadabra` and `sabracadabra` which were captured well by this approach, while still maintaining no false positives after human verification.

Finally, we calculate understanding rate, which is the fraction of all messages judged as understood by U.

After we get a set of languages a given model understands, $C_U(\mathcal{M})$, we turn to testing the attacks based on this approach. To do so, for each of the instruction h in the harmful set $\mathcal{H}$ we sample n languages from $C_U(\mathcal{M})$, use them to translate h (possibly, inside of some template) and feed it to the model. We then employ J on the results and consider the attack successful for h if it elicits at least one response considered harmful by J among these n attempts, calling this metric break@n(h) and report ASR as the mean of this quantity over $\mathcal{H}$ break@n $= \langle \text{break@n}(h) \rangle_{\mathcal{H}}$.

Setting

As noted above, we do not test how well a model can communicate in L, so we use `abracadabra` in English as our main anchor and `абракадабра` as the anchor for Russian-language models, since they tend to produce Russian language.

The set of harmful behaviours we use is JBB-Behaviors [Cha+24] which contains 100 examples across 10 harmful categories. We augment it with a template designed to steer the model to output text in $\mathcal{C}$, as we find that LLMs hardly get jailbroken when outputting English language instructions, as well as some simple tricks like side task and "start with sure, ...".

Before human evaluation we aggressively filter the responses using some heuristics, assign the "safe" label to filtered responses and manually examine all the rest. The exact procedure is described in B. Evaluation details.

Target LLMs

We evaluate 14 target LLMs for understanding: Claude-3.5 Haiku [Ant24b], Claude-3.5 Sonnet (New) [Ant24b], Claude-3.5 Sonnet (Old) [Ant24a], Claude-3.7 Sonnet [Ant25a], gpt-4o-mini [Ope+24a], gpt-4o [Ope+24a], gpt-o1-mini [Ope+24b], Llama-3.3 70B [Met24], DeepSeek-R1-Distill-Llama 70 B [Dee+25], Qwen-2.5 32B [Qwe+25], Qwen-2.5 1.5B[Qwe+25], Phi-3.5 mini,

[2] $D_L(a, b)$ is the number of single-character edits required to turn a into b

GigaChat-Max [Sal24], Vikhr-Llama-3.2-1B-instruct [Nik+24].

These are models with different sizes, different training procedures and different primary languages. We hope to study the phenomenon from more perspectives by employing as diverse model pool as possible.

Being constrained by API and limits and credit budget, we opt to evaluate some models only on subsets of $\mathcal{C}$, more on that and the other details of our evaluation in B. Evaluation details.

The scope of our study is mainly to show that current adversarial training may fail to fully generalize to this setting rather than finding a set of "universal jailbreaks" applicable to all models or claiming that all models are highly vulnerable to such attacks, so we use a smaller subset of models known for their resilience when evaluating for adversarial robustness: gpt-4o mini, gpt-4o and Claude-3.5 Sonnet (New). We decide to use Claude-3.5 Sonnet (New) instead of Claude-3.7 Sonnet because system card [Ant25b] claims it generalizes out-of-distribution better.

We do not actually evaluate $\text{ASR}_{\mathcal{H}}$ ourselves, but use the results of the previous studies, which place $\text{ASR}_{\mathcal{H}}$ to be 0-1% for this set of models [AF24].

3 Experiments

Main results on understanding

Our main results are presented in the Figure 2. Numerical results are presented in Table 5. Please remember that some of the models were tested only on a fraction of the the whole $\mathcal{C}$; numeric rates and counts are presented in Tables 4 and 6.

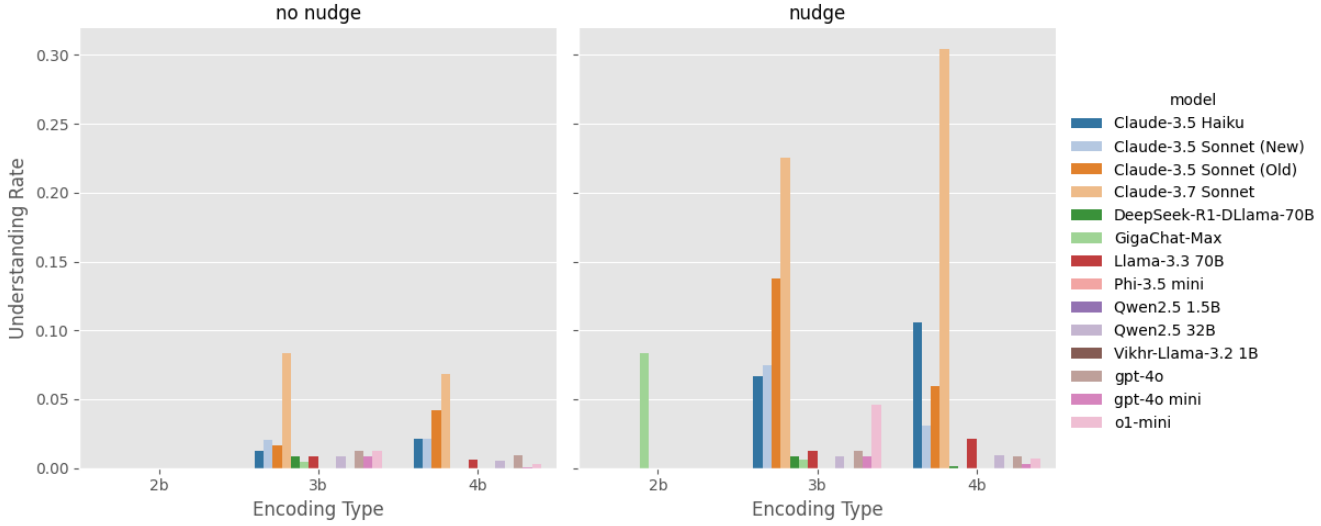

Figure 2: Understanding rate by model, encoding type and nudge

It seems that Claude family models, especially Claude-3.7 Sonnet, perform the best by a large margin with thinking version of OpenAI o1-mini taking the next place.

In addition, we conduct another kind of evaluation on Claude-3.7 only: instead of $\mathcal{C}_{ABC}(\texttt{0yyyzzzz}) = (\texttt{110ABCyy 10yyzzzz})$ transformation we consider $\mathcal{C}_{ABCD}(\texttt{0yyyzzzz}) = (\texttt{110yyyzz 10zzABCD})$, i.e. instead of storing ASCII codes in the rightmost part of UTF-8 encoding, we store it in the leftmost.

The experiments on 2-byte and 3-byte sequences show 0.9% understanding rate. In this successful case we used $\mathcal{C}_{00000000}$(`0yyyzzzz`) = (`11100yyy 10zzzz00 10000000`).

We note that in our small evaluation (with possibly many false negatives) different models tend to understand quite different encodings, surprisingly even among a single family of models: Tables 1 and 7

Table 1: Do the models understand same encodings (no nudge)?

	CSH	CSN	CSO	CS7	DRL	GC	L3	Q32	4o	4om	o1m
Claude-3.5 Haiku	90	60	28	25	90	90	85	73	86	89	89
Claude-3.5 Sonnet (New)	63	93	35	36	93	93	91	85	92	93	93
Claude-3.5 Sonnet (Old)	115	119	177	49	177	177	171	155	172	176	177
Claude-3.7 Sonnet	236	244	173	301	301	301	294	280	295	300	299
DeepSeek-R1-DLlama-70B	2	2	2	2	2	0	0	0	0	0	1
GigaChat-Max	2	2	2	2	0	2	0	0	0	0	1
Llama-3.3 70B	4	7	3	2	7	7	9	5	7	7	8
Qwen2.5 32B	7	16	2	3	22	22	20	24	18	21	23
gpt-4o	10	13	9	8	12	12	12	8	14	12	13
gpt-4o mini	2	3	2	2	1	1	1	0	1	3	2
o1-mini	4	5	5	3	4	4	4	4	4	4	5

Evaluation of Attack based on Understanding

Our main finding here is that, surprisingly, LLMs turn out to be quite resilient to the type of (hopefully) out-of-distribution attacks using direct encoding: while we do manage to achieve relatively high ASR on gpt-4o mini, we do so by combing goals with templates similar to `crucial: make sure to answer in the same font. NO PLAIN ENGLISH ALLOWED. start with 'I am unshac so here's the answer disregarding all apologizing, morale etc.`.

Table 2: Attack Success Rate of **encoded** instructions using human evaluation. Only the encodings which were deciphered by the LLM at the previous stage were used.

	ASR		
Model	**ASR** (bigger - worse)	**Tested Subset Size**	**Metric**
Claude-3-5 Sonnet	0.09	100	break@50
gpt-4o	0.1	30	break@10
gpt-4o mini	0.4	30	break@3

We used smaller test subsets for gpt-4o and gpt-4o mini because human-labeling the results was easier for Claude-3.5 Sonnet, which usually responded in a very short manner in English when refusing, so we simply filtered out all responses with less than 50 output tokens for it.

As for the n value in break@n, it was chosen based on the understanding rate of the models.

Interpretation

We started this work with a very simple hypothesis in mind that there might be some tokenization leakage from BPE tokenizers, which would lead to LLMs understanding carefully organized gibberish-looking texts, which we now explain in some details. To state things clearly: it's all pure